Data Analytics Project Report

Customer Churn Analysis & Customer Intelligence

End-to-end churn analytics for an OTT subscription business using SQL and Python

Author	Gaurav Kalamkhede
Tools	SQL (SQLite), Python, pandas, NumPy, Matplotlib, Seaborn
Repository	github.com/gauravkal006/Customer-Churn-Analysis
Data snapshot	31 December 2025

Contents

1. Executive summary
2. Business problem and objectives
3. Data and tools
4. Project workflow
5. Data import: Excel → SQLite → pandas
6. Data quality checks and cleaning
7. Feature engineering and joins
8. KPI results
9. Segment analysis and visualisations
10. SQL validation
11. Key insights
12. Recommendations
13. Limitations and next steps
14. Project deliverables
15. Appendix: resume highlights

1. Executive summary

I analysed 21 subscribers of an OTT streaming platform across three relational tables (customer demographics, subscriptions and support tickets) to find **who** churns, **why** and **when**. After building a SQLite database from the raw export, cleaning and joining the tables in pandas and engineering churn features, I calculated 20+ KPIs, checked them in SQL and visualised the main drivers.

Measure	Value	Context
Churn rate	28.6%	6 of 21 customers
Retention rate	71.4%	Customers still active
MRR lost	Rs 73.94	18.7% of monthly revenue
CLTV lost	Rs 2,047	Lifetime value of churned users
Monthly-contract churn	55.6%	vs 8.3% for annual
Escalation ↔ churn corr.	0.88	100% of escalated users churned

In short, churn is driven by **monthly contracts**, **unresolved escalations** and **low-intent referral sign-ups**. It is concentrated in the Basic plan and in low-CLTV customers. Moving monthly users to annual plans and treating every escalation as a churn alert are the two highest-impact actions.

2. Business problem and objectives

In a competitive OTT market (Netflix, Prime Video, Hotstar), acquiring a subscriber costs several times more than keeping one. Leadership wants a data-backed view of churn so that retention budget goes where it protects the most revenue.

Who	Which customers left, and which active customers are likely to leave next?
Why	Which plans, contracts, channels and support experiences go with churn?
When	In which months, and after how much tenure, does churn happen?

Objectives

- Build a reproducible pipeline from the raw Excel export to an analysis-ready customer master table.
- Quantify churn, retention, revenue loss and revenue at risk.
- Find the segments and behaviours that drive churn.
- Turn the findings into prioritised, measurable retention actions.

Churn definition: a subscriber has churned when the subscription has a `cancellation_date`. Tenure and age are measured at a fixed snapshot date (31-Dec-2025), so the results can be reproduced.

3. Data and tools

Table	Rows	Key columns
db_customer	21	customerid, name, country, state, gender, dob, interests, pincode
db_subscription	21	subscription_start_date, subscription_type, renewal_date, plan_type, contract_type, cancellation_date, cancellation_reason, monthly_charges, cltv, churn_score
db_support	9	complaint_date, escalations, csat_score, col_1, comment

Area	Tools used
Storage and querying	SQLite, SQL (SELECT, JOIN, CASE WHEN, GROUP BY, HAVING, PRAGMA)
Data wrangling	Python, pandas, NumPy
Visualisation	Matplotlib, Seaborn
Environment	Jupyter Notebook, VS Code, Git/GitHub

4. Project workflow

#	Step	What I did
1	Import	Excel workbook → SQLite tables → pandas via SQL
2	Profile	Nulls, duplicates and label checks per table
3	Clean	Types, renames, drops, standardisation, imputation, de-duplication
4	Engineer	Churn flag, tenure, age, complaints, escalation, risk tier
5	Join	Subscription ⋈ customer ⋈ support with row-count assertion
6	Analyse	20+ KPIs, segment analysis, pivot tables
7	Validate	KPIs recomputed in pure SQL
8	Visualise	10 charts → insights → recommendations



Figure 1. Project structure

5. Data import: Excel → SQLite → pandas

The source data came as one Excel workbook with a sheet per table. I loaded every sheet into its own SQLite table so the analysis reads from a relational database the way it would from a production warehouse. I then inspected the schema with `sqlite_master` and `PRAGMA table_info`, and pulled each table into pandas with SQL.

customer_churn_data_raw.xlsx — Excel

	A	B	C	D	E	F	G	H	I	J	K
1	customerid	subscription_start_date	subscription_type	renewal_date	plan_type	contract_type	cancellation_date	cancellation_reason	monthly_charges	cltv	churn_score
2	0002-ORFBO	2021-03-15	Refferal	2025-03-15	Standard	Annual			13.99	627	12
3	0003-MKNFE	2020-08-01	Paid	2024-08-01	Premium	Annual	2024-09-10	Switched to competitor	12.99	1150	91
4	0004-TLHLJ	2022-11-20	Organic	2025-11-20	Basic	Monthly			6.99	210	34
5	0011-IGKFF	2019-05-10	Paid	2025-05-10	Premium	Annual			22.99	1725	8
6	0013-EXCHZ	2023-01-05	Refferal	2024-01-05	Standard	Monthly	2024-02-28	Too expensive	13.99	195	88
7	0013-MHZWF	2022-06-18	Paid	2025-06-18	Standard	Annual			17.99	720	22
8	0013-SMEOE	2021-09-30	Refferal	2024-09-30	Basic	Monthly	2024-11-15	Not enough content	8.99	230	79
9	0014-BMAQU	2020-02-14	Organic	2025-02-14	Premium	Annual			22.99	1840	5
10	0015-UOCOJ	2023-07-22	Organic	2024-07-22	Standard	Monthly			13.99	240	34
11	0016-QLJIS	2022-04-03	Organic	2025-04-03	Basic	Annual			6.99	335	41
12	0017-DINOC	2021-12-01	Organic	2025-12-01	Premium	Annual			22.99	1610	14
13	0017-IUDMW	2023-03-17	Refferal	2024-03-17	Standard	Monthly	2024-05-01	Poor streaming quality	7.99	270	83

db_subscription

Figure 2. Raw data: the subscription sheet of the source Excel workbook

```
In [2]: # build the SQLite database from the raw workbook (one table per sheet)
        with sqlite3.connect(DB_PATH) as conn:
            for sheet in pd.ExcelFile(RAW_XLSX).sheet_names:
                pd.read_excel(RAW_XLSX, sheet_name=sheet).to_sql(sheet, conn, if_exists="replace", index=False)
                print(f"Loaded sheet -> table: {sheet}")

Loaded sheet -> table: db_customer
Loaded sheet -> table: db_subscription
Loaded sheet -> table: db_support
```

Figure 3. Loading each Excel sheet into its own SQLite table

```
In [3]: # list tables and their columns using SQLite metadata (sqlite_master + PRAGMA)
        conn = sqlite3.connect(DB_PATH)
        tables = pd.read_sql("SELECT name FROM sqlite_master WHERE type='table';", conn)

        for table_name in tables["name"]:
            cols = pd.read_sql(f"PRAGMA table_info({table_name});", conn)["name"].tolist()
            rows = pd.read_sql(f"SELECT COUNT(*) AS n FROM {table_name};", conn)["n"][0]
            print(f"{table_name:<16} rows={rows:<3} columns={cols}")

db_customer      rows=21  columns=['customerid', 'name', 'country', 'state', 'gender', 'dob', 'interests', 'pincode']
db_subscription  rows=21  columns=['customerid', 'subscription_start_date', 'subscription_type', 'renewal_date', 'plan_type',
'contract_type', 'cancellation_date', 'cancellation_reason', 'monthly_charges', 'cltv', 'churn_score']
db_support       rows=9   columns=['customerid', 'complaint_date', 'escalations', 'csat_score', 'col_1', 'comment']
```

Figure 4. Reading table metadata with sqlite_master and PRAGMA table_info

In [5]:
```python
def quality_report(df, name):
    report = pd.DataFrame({
        "dtype": df.dtypes.astype(str),
        "nulls": df.isna().sum(),
        "null_%": (df.isna().mean() * 100).round(1),
        "unique": df.nunique(),
    })
    print(f"{name}: {df.shape[0]} rows | duplicate customerid rows = {df['customerid'].duplicated().sum()}")
    return report

quality_report(df_customer, "db_customer")
```

```
db_customer: 21 rows | duplicate customerid rows = 0
```

Out[5]:

	dtype	nulls	null_%	unique
customerid	object	0	0.0	21
name	object	0	0.0	21
country	object	3	14.3	2
state	object	0	0.0	9
gender	object	0	0.0	4
dob	object	0	0.0	21
interests	object	17	81.0	4
pincode	object	21	100.0	0

Figure 5. Data-quality profile of the customer table

Issues found

Table	Issue	Fix
customer	`name` is ambiguous	rename to `customer_name`
customer	`interests` (81% null) and `pincode` (100% null) carry no information	drop
customer	`dob` stored as text	convert to datetime
customer	gender has 4 labels (`Male` , `Female` , `Men` , `Women`)	standardise to Male/Female
customer	3 rows missing `country`	infer from the state → country mapping
subscription	dates stored as text, typo `Refferal`	convert to datetime, fix label
support	`col_1` empty, `comment` free text with typos	drop
support	customers with more than one complaint → would duplicate rows on join	aggregate to one row per customer

Figure 6. Data issues found and the fix applied to each

In [11]:
```python
# every state belongs to exactly one country -> build the mapping from complete rows and fill the gaps
state_country_map = df_customer.dropna(subset=["country"]).set_index("state")["country"].to_dict()
df_customer["country"] = df_customer["country"].fillna(df_customer["state"].map(state_country_map))

print("Missing country after fix:", df_customer["country"].isna().sum())
df_customer.head()
```

```
Missing country after fix: 0
```

Out[11]:

	customerid	customer_name	country	state	gender	dob
0	0002-ORFBO	Keshav	India	Maharashtra	Male	1982-04-12
1	0003-MKNFE	Raghav	India	Karnataka	Male	1995-11-23
2	0004-TLHLJ	Lalita	India	Delhi	Female	1978-02-15
3	0011-IGKFF	Mohan	India	Nagaland	Male	2001-08-30
4	0013-EXCHZ	Mira	India	Delhi	Female	1990-05-05

Figure 7. Filling missing country values from the state → country mapping

In [14]:
```
# 9 complaint rows but only 7 customers -> joining as-is would duplicate subscribers.
# Aggregate to one row per customer: total complaints, escalated at least once, latest CSAT.
df_support["complaint_count"] = df_support.groupby("customerid")["customerid"].transform("count")
df_support["escalations"] = (df_support.groupby("customerid")["escalations"]
                                       .transform(lambda s: "Y" if (s == "Y").any() else "N"))
df_support = df_support.sort_values("complaint_date").drop_duplicates("customerid", keep="last")

print("rows:", len(df_support), "| unique customers:", df_support["customerid"].nunique())
df_support
```

```
rows: 7 | unique customers: 7
```

Out[14]:

	customerid	complaint_date	escalations	csat_score	complaint_count
2	0013-EXCHZ	2024-01-20	Y	20	1
5	0017-IUDMW	2024-04-10	Y	25	1
1	0003-MKNFE	2024-08-28	Y	10	2
8	0022-TCJCI	2024-09-14	Y	90	2
6	0019-EFAEP	2024-09-27	Y	30	1
4	0013-SMEOE	2024-11-01	N	30	1
3	0013-MHZWF	2025-03-18	N	90	1

Figure 8. Collapsing duplicate support tickets to one row per customer

In [15]:
```
# churn flag: a customer has churned if a cancellation date exists
df_subscription["churn_flag"] = np.where(df_subscription["cancellation_date"].notna(), 1, 0)

# LEFT JOIN subscription -> customer -> support (subscription is the customer base)
print("subscription rows before join:", len(df_subscription))
df = (df_subscription
        .merge(df_customer, on="customerid", how="left")
        .merge(df_support, on="customerid", how="left"))
print("master rows after join       :", len(df))
assert len(df) == len(df_subscription), "join created duplicate rows"
```

```
subscription rows before join: 21
master rows after join       : 21
```

Figure 9. Joining the three tables with a row-count check

In [16]:

```python
# tenure: days from start until cancellation (churned) or until the snapshot date (active)
end_date = df["cancellation_date"].fillna(SNAPSHOT_DATE)
df["tenure_days"] = (end_date - df["subscription_start_date"]).dt.days

# customer age and age band
df["age"] = ((SNAPSHOT_DATE - df["dob"]).dt.days // 365.25).astype(int)
df["age_group"] = pd.cut(df["age"], bins=[0, 25, 35, 45, 100], labels=["<25", "25-34", "35-44", "45+"], right=False)

# support features: customers without complaints get 0
df["complaint_count"] = df["complaint_count"].fillna(0).astype(int)
df["escalation_flag"] = np.where(df["escalations"] == "Y", 1, 0)

# churn-risk tier from the model churn score
df["churn_risk"] = np.select(
    [df["churn_score"] < 50, df["churn_score"] >= 70],
    ["Low", "Medium", "High"], default="Unknown")

df["cancellation_month"] = df["cancellation_date"].dt.to_period("M")

df[["customerid", "plan_type", "contract_type", "churn_flag", "tenure_days", "age", "age_group",
    "complaint_count", "escalation_flag", "churn_score", "churn_risk"]].head(8)
```

Out[16]:

	customerid	plan_type	contract_type	churn_flag	tenure_days	age	age_group	complaint_count	escalation_flag	churn_score	churn_ri
0	0002-ORFBO	Standard	Annual	0	1752	43	35-44	0	0	12	Lo
1	0003-MKNFE	Premium	Annual	1	1501	30	25-34	2	1	91	Hi
2	0004-TLHLJ	Basic	Monthly	0	1137	47	45+	0	0	34	Lo
3	0011-IGKFF	Premium	Annual	0	2427	24	<25	0	0	8	Lo
4	0013-EXCHZ	Standard	Monthly	1	419	35	35-44	1	1	88	Hi
5	0013-MHZWF	Standard	Annual	0	1292	37	35-44	1	0	22	Lo
6	0013-SMEOE	Basic	Monthly	1	1142	49	45+	1	0	79	Hi
7	0014-BMAQU	Premium	Annual	0	2147	26	25-34	0	0	5	Lo

Figure 10. Feature engineering: tenure, age, complaints, escalation, churn-risk tier

6. Data quality checks and cleaning

I profiled every table for nulls, duplicates and inconsistent labels before changing anything. The issues and the fix applied to each:

Table	Issue found	Fix applied
db_customer	Ambiguous column name `name`	Renamed to customer_name
db_customer	interests 81% null, pincode 100% null	Dropped (no information)
db_customer	dob stored as text	Converted to datetime
db_customer	Gender labels Male / Female / Men / Women	Standardised to Male / Female
db_customer	3 rows missing country	Filled from the state → country mapping
db_subscription	Three date columns stored as text	Converted to datetime
db_subscription	Typo 'Refferal' in subscription_type	Corrected to 'Referral'
db_support	col_1 empty; comment free text with typos	Dropped
db_support	9 tickets for 7 customers (duplicates on join)	Aggregated: complaint_count, any-escalation flag, latest CSAT

The most important fix was in the support table. Two customers had two tickets each, so a direct join would have produced 23 rows for 21 subscribers and inflated every KPI. I collapsed the tickets to one row per customer, keeping the complaint count, whether the customer was ever escalated, and the latest CSAT score.

7. Feature engineering and joins

Feature	Logic
churn_flag	1 if cancellation_date is present, else 0
tenure_days	Days from subscription start to cancellation date (churned) or snapshot date 31-Dec-2025 (active)
age / age_group	Age at the snapshot date, banded <25, 25-34, 35-44, 45+
complaint_count	Number of support tickets per customer (0 if none)
escalation_flag	1 if any of the customer's tickets was escalated
churn_risk	Low (<50), Medium (50-69), High (≥70) from churn_score
cancellation_month	Year-month of cancellation, used for the trend

I left-joined the tables on `customerid` with subscription as the base table, then asserted that the joined table still had exactly 21 rows. The clean master table is exported to `data/processed/churn_master_clean.csv`.

8. KPI results

KPI	Value	Definition
Total customers	21	COUNT(customerid)
Churned customers	6	COUNT where cancellation_date IS NOT NULL
Churn rate	28.57%	churned / total customers
Retention rate	71.43%	100 − churn rate
ARPU (all customers)	Rs 18.85	AVG(monthly_charges)
ARPU (active customers)	Rs 21.46	AVG(monthly_charges) of active customers
Average tenure	1,355 days	AVG(cancellation date or snapshot date − start date)
Total MRR	Rs 395.79	SUM(monthly_charges)
MRR lost to churn	Rs 73.94 (18.68%)	SUM(monthly_charges) of churned customers
CLTV lost	Rs 2,047	SUM(cltv) of churned customers
Avg CLTV churned vs retained	Rs 341.17 vs Rs 1016.47	AVG(cltv) by churn_flag
Escalation rate	23.81%	customers with ≥1 escalation / total customers
Avg complaints per customer	0.43	COUNT(complaints) / COUNT(DISTINCT customerid)
Correlation escalation vs churn	0.88	Pearson corr(escalation_flag, churn_flag)
Churn rate with / without escalation	100% / 6.25%	churn rate split by escalation_flag
Monthly vs annual contract churn	55.56% / 8.33%	churn rate GROUP BY contract_type
Active customers at Medium/High risk	2	active AND churn_score ≥ 50
MRR / CLTV at risk	Rs 34.98 / Rs 1,190	SUM over at-risk active customers
Average customer age	35.9 years	snapshot date − dob

In [18]:

```
churned = df[df["churn_flag"] == 1]
active = df[df["churn_flag"] == 0]

churn_rate = df["churn_flag"].mean() * 100
retention_rate = 100 - churn_rate
arpu = df["monthly_charges"].mean()
arpu_active = active["monthly_charges"].mean()
avg_tenure = df["tenure_days"].mean()
total_mrr = df["monthly_charges"].sum()
mrr_lost = churned["monthly_charges"].sum()
cltv_lost = churned["cltv"].sum()
escalation_rate = df["escalation_flag"].mean() * 100
avg_complaints = df["complaint_count"].sum() / df["customerid"].nunique()
corr_esc_churn = df["escalation_flag"].corr(df["churn_flag"])

kpis = pd.Series({
    "Total customers": len(df),
    "Churned customers": len(churned),
    "Churn rate (%)": round(churn_rate, 2),
    "Retention rate (%)": round(retention_rate, 2),
    "ARPU - all customers (Rs)": round(arpu, 2),
    "ARPU - active customers (Rs)": round(arpu_active, 2),
    "Avg tenure (days)": round(avg_tenure),
    "Total monthly revenue / MRR (Rs)": round(total_mrr, 2),
    "MRR lost to churn (Rs)": round(mrr_lost, 2),
    "MRR lost (%)": round(mrr_lost / total_mrr * 100, 2),
    "CLTV lost (Rs)": cltv_lost,
    "Escalation rate (%)": round(escalation_rate, 2),
    "Avg complaints per customer": round(avg_complaints, 2),
    "Correlation escalation vs churn": round(corr_esc_churn, 2),
}, dtype=object)
kpis.to_frame("value")
```

Out[18]:

	value
Total customers	21
Churned customers	6
Churn rate (%)	28.57
Retention rate (%)	71.43
ARPU - all customers (Rs)	18.85
ARPU - active customers (Rs)	21.46
Avg tenure (days)	1355
Total monthly revenue / MRR (Rs)	395.79
MRR lost to churn (Rs)	73.94
MRR lost (%)	18.68
CLTV lost (Rs)	2047
Escalation rate (%)	23.81
Avg complaints per customer	0.43
Correlation escalation vs churn	0.88

Figure 11. KPI calculation output

In [21]:

```
# revenue AT RISK: active customers whose churn score puts them in the Medium/High tier
at_risk = active[active["churn_risk"].isin(["High", "Medium"])]
print(f"Active customers at risk : {len(at_risk)}")
print(f"MRR at risk (Rs)         : {at_risk['monthly_charges'].sum():.2f}")
print(f"CLTV at risk (Rs)        : {at_risk['cltv'].sum()}")
at_risk[["customerid", "customer_name", "plan_type", "contract_type", "monthly_charges", "cltv", "churn_score", "churn_risk"]]
```

```
Active customers at risk : 2
MRR at risk (Rs)         : 34.98
CLTV at risk (Rs)        : 1190
```

Out[21]:

	customerid	customer_name	plan_type	contract_type	monthly_charges	cltv	churn_score	churn_risk
15	0020-INWCK	Parvati	Standard	Monthly	13.99	640	58	Medium
16	0020-JDNXP	Rikim	Premium	Annual	20.99	550	62	Medium

Figure 12. Active customers at risk and the revenue they represent

In [30]:
```
conn = sqlite3.connect(DB_PATH)

kpi_sql = """
SELECT COUNT(*)                                                    AS customers,
       SUM(CASE WHEN cancellation_date IS NOT NULL THEN 1 ELSE 0 END) AS churned,
       ROUND(100.0 * AVG(CASE WHEN cancellation_date IS NOT NULL THEN 1 ELSE 0 END), 2) AS churn_rate_pct,
       ROUND(AVG(monthly_charges), 2)                              AS arpu,
       ROUND(SUM(CASE WHEN cancellation_date IS NOT NULL THEN monthly_charges END), 2) AS mrr_lost,
       SUM(CASE WHEN cancellation_date IS NOT NULL THEN cltv END)    AS cltv_lost
FROM db_subscription;
"""
pd.read_sql(kpi_sql, conn)
```

Out[30]:

	customers	churned	churn_rate_pct	arpu	mrr_lost	cltv_lost
0	21	6	28.57	18.85	73.94	2047

Figure 13. Recomputing the headline KPIs in pure SQL

In [31]:
```
plan_sql = """
SELECT s.plan_type,
       s.contract_type,
       COUNT(*)                                                    AS customers,
       ROUND(100.0 * AVG(CASE WHEN s.cancellation_date IS NOT NULL THEN 1 ELSE 0 END), 1) AS churn_rate_pct,
       ROUND(SUM(s.monthly_charges), 2)                            AS monthly_revenue
FROM db_subscription s
GROUP BY s.plan_type, s.contract_type
ORDER BY churn_rate_pct DESC;
"""
pd.read_sql(plan_sql, conn)
```

Out[31]:

	plan_type	contract_type	customers	churn_rate_pct	monthly_revenue
0	Basic	Monthly	4	75.0	45.96
1	Standard	Monthly	5	40.0	63.95
2	Premium	Annual	7	14.3	218.93
3	Basic	Annual	1	0.0	6.99
4	Standard	Annual	4	0.0	59.96

Figure 14. SQL: churn by plan and contract

In [45]:
```
pd.pivot_table(df, index="plan_type", columns="contract_type", values="churn_flag", aggfunc="mean", margins=True).mul(100).rou
```

Out[45]:

contract_type	Annual	Monthly	All
plan_type			
Basic	0.0	75.0	60.0
Premium	14.3	NaN	14.3
Standard	0.0	40.0	22.2
All	8.3	55.6	28.6

Figure 15. Pivot table: churn rate by plan × contract

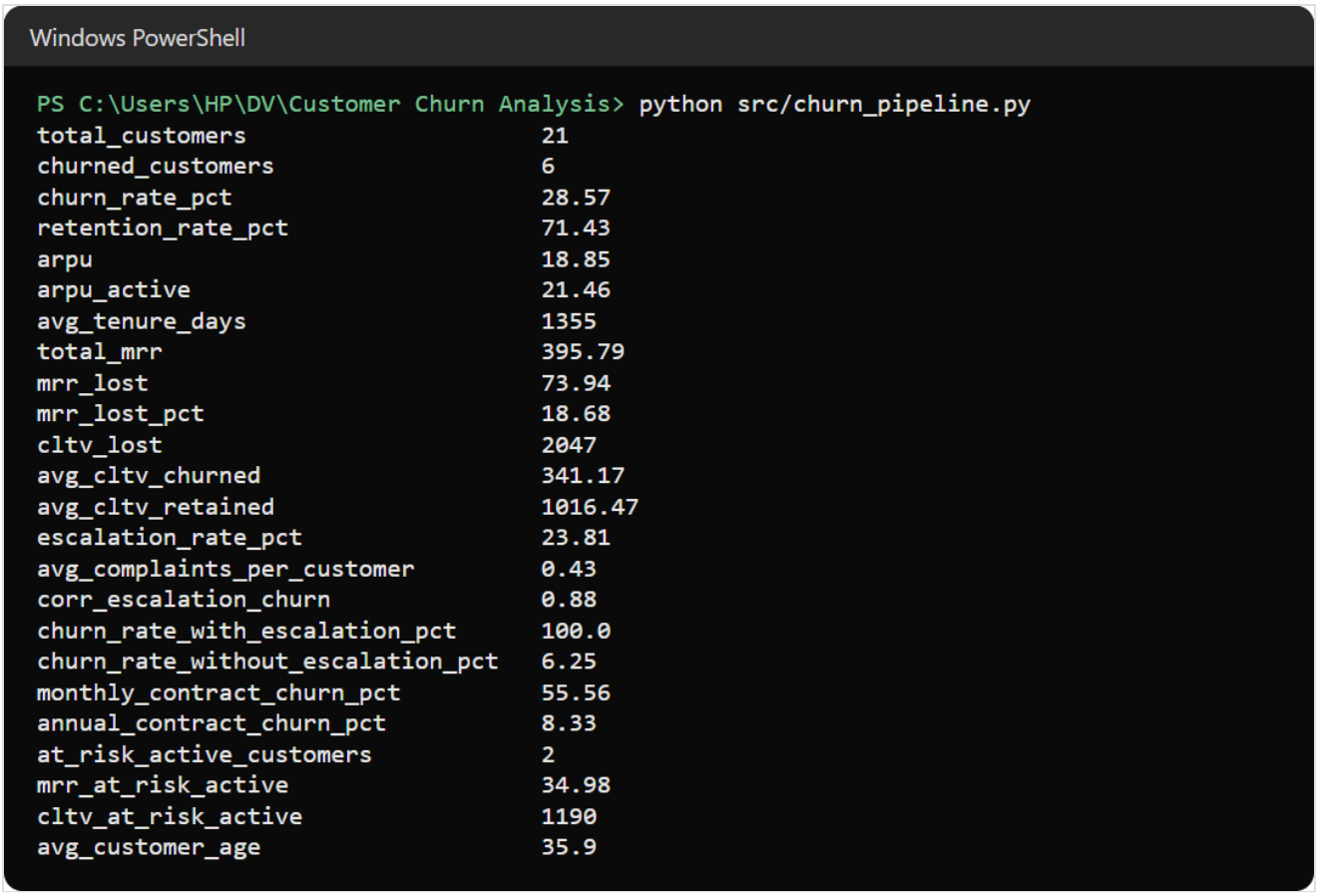


Figure 16. Running the reusable pipeline script from the terminal

9. Segment analysis and visualisations

9.1 Plan, contract and acquisition channel

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
Basic	5	3	60.0%	Rs 52.95	Rs 977
Standard	9	2	22.2%	Rs 123.91	Rs 5,302
Premium	7	1	14.3%	Rs 218.93	Rs 11,015

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
Monthly	9	5	55.6%	Rs 109.91	Rs 2,777
Annual	12	1	8.3%	Rs 285.88	Rs 14,517

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
Referral	6	5	83.3%	Rs 74.94	Rs 1,524
Paid	6	1	16.7%	Rs 174.94	Rs 7,410
Organic	9	0	0.0%	Rs 145.91	Rs 8,360

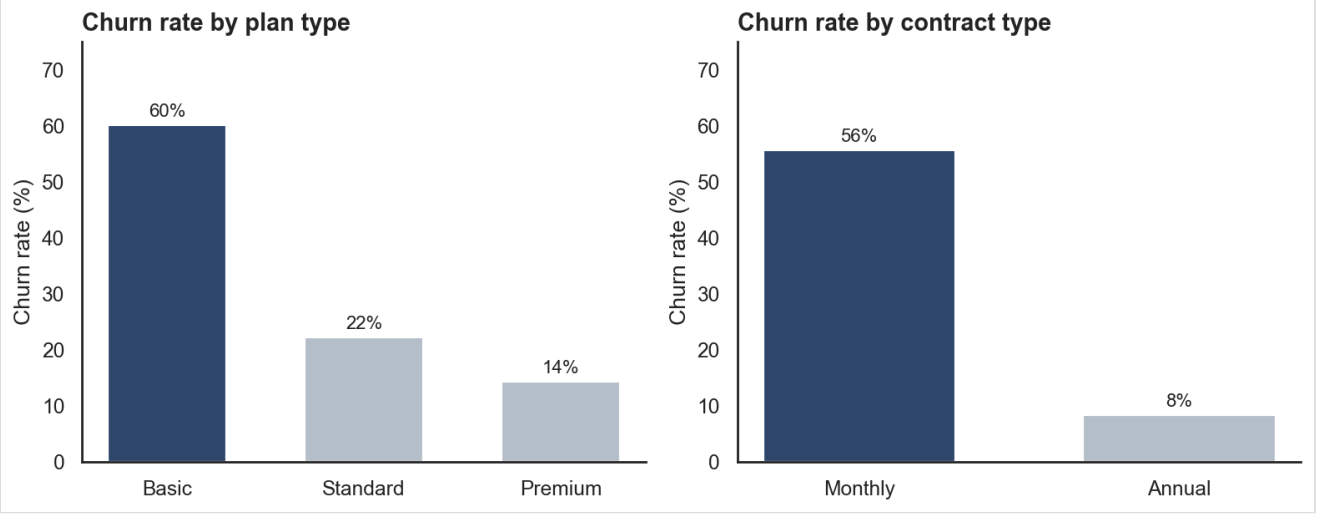


Figure 17. Churn rate by plan type and contract type

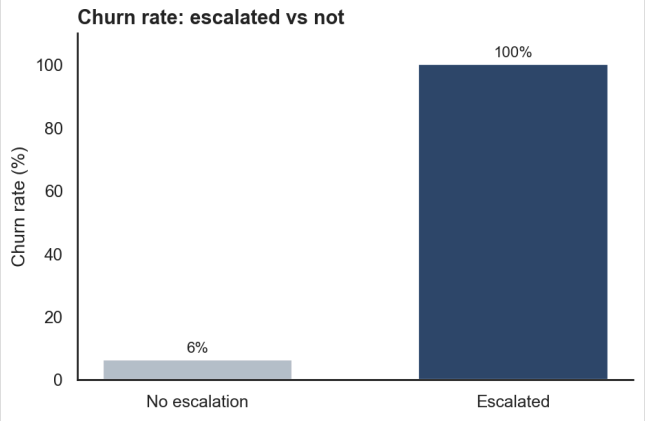


Figure 18. Churn rate for escalated vs non-escalated customers

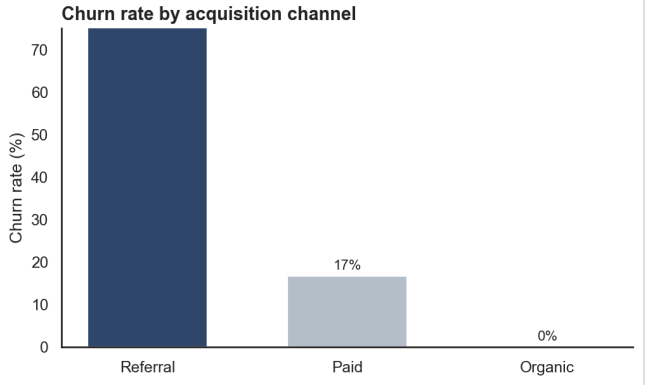


Figure 19. Churn rate by acquisition channel

9.2 When churn happens and where

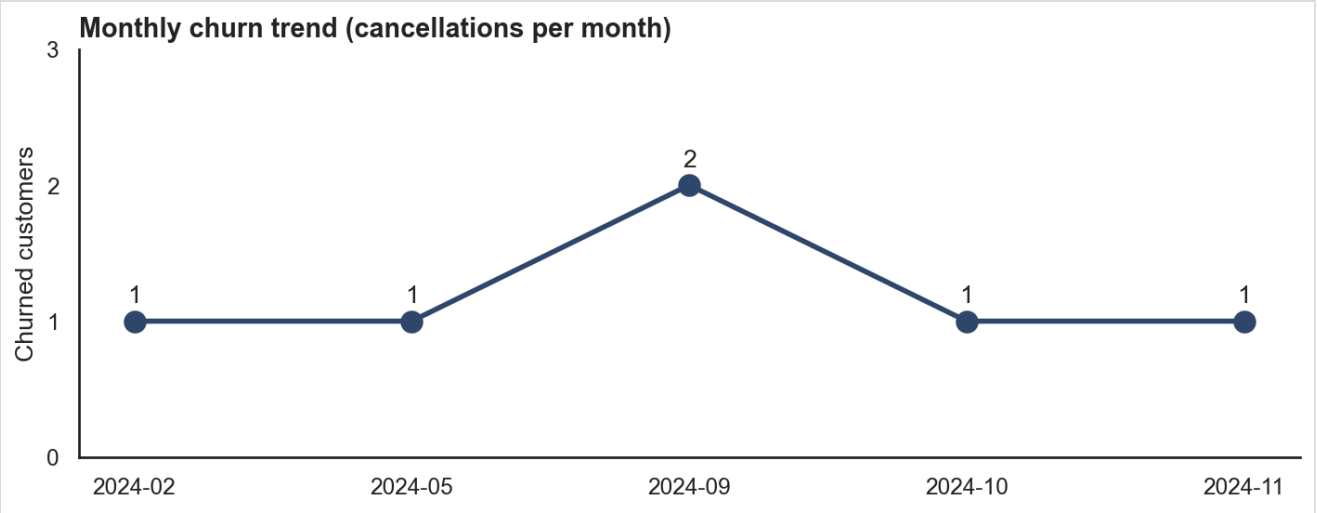


Figure 20. Monthly churn trend

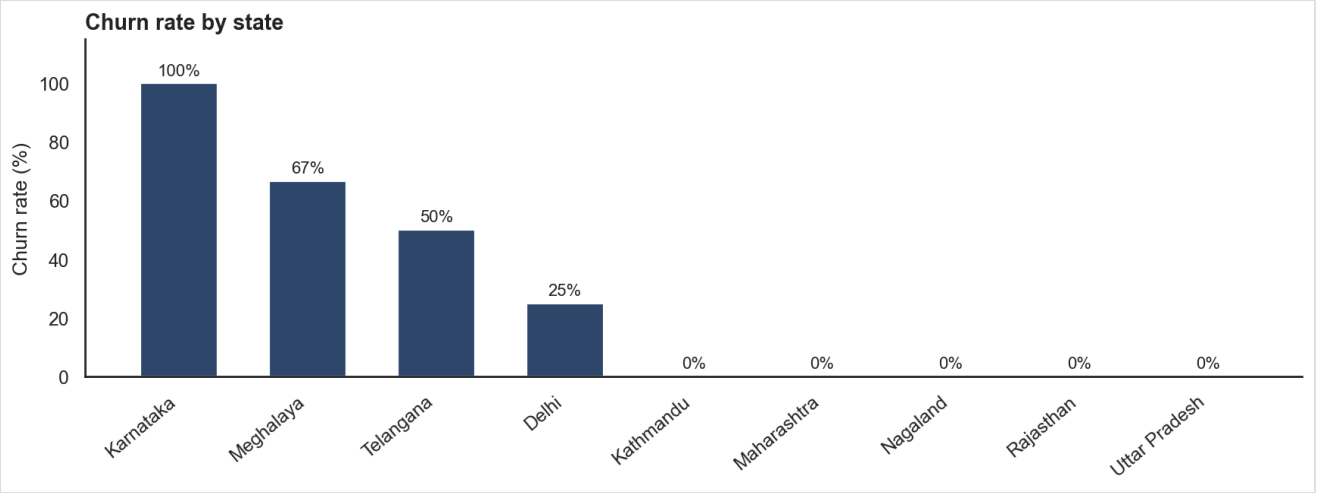


Figure 21. Churn rate by state

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
Karnataka	2	2	100.0%	Rs 20.98	Rs 1,420
Meghalaya	3	2	66.7%	Rs 42.97	Rs 940
Telangana	2	1	50.0%	Rs 30.98	Rs 1,022
Delhi	4	1	25.0%	Rs 52.96	Rs 1,765
Kathmandu	2	0	0.0%	Rs 20.98	Rs 575
Maharashtra	3	0	0.0%	Rs 50.97	Rs 3,027
Nagaland	1	0	0.0%	Rs 22.99	Rs 1,725
Rajasthan	2	0	0.0%	Rs 36.98	Rs 2,680
Uttar Pradesh	2	0	0.0%	Rs 115.98	Rs 4,140

9.3 Why customers leave

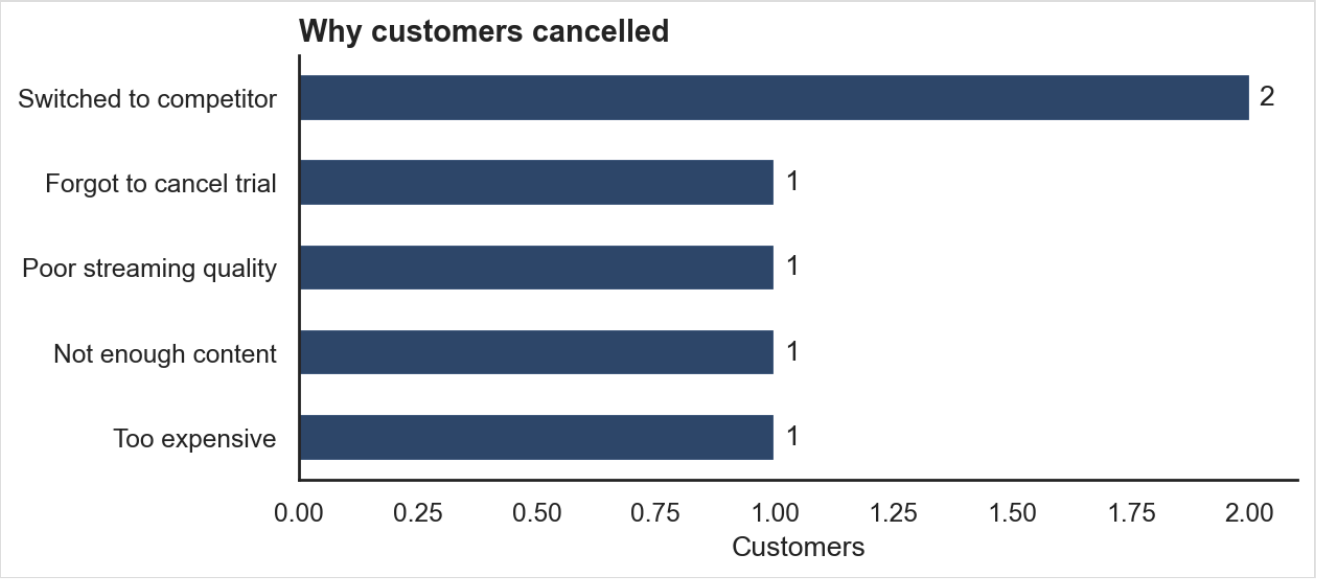


Figure 22. Cancellation reasons

9.4 Risk tiers and lifetime value

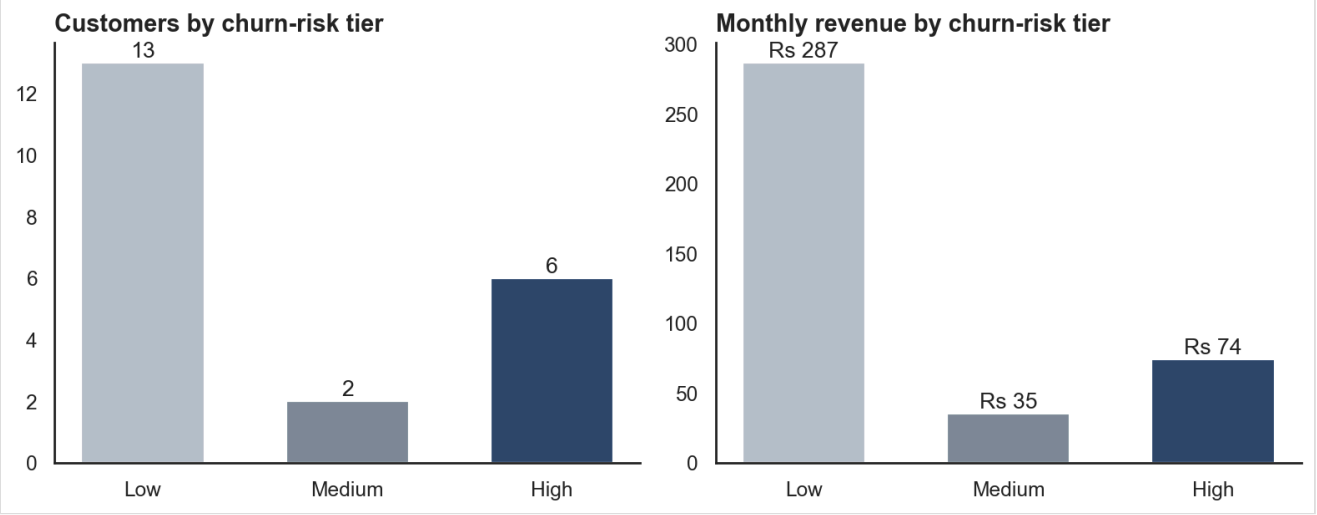


Figure 23. Customers and revenue by churn-risk tier

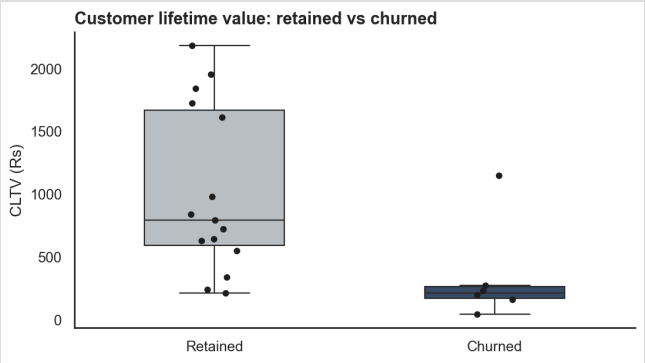


Figure 24. CLTV of retained vs churned customers

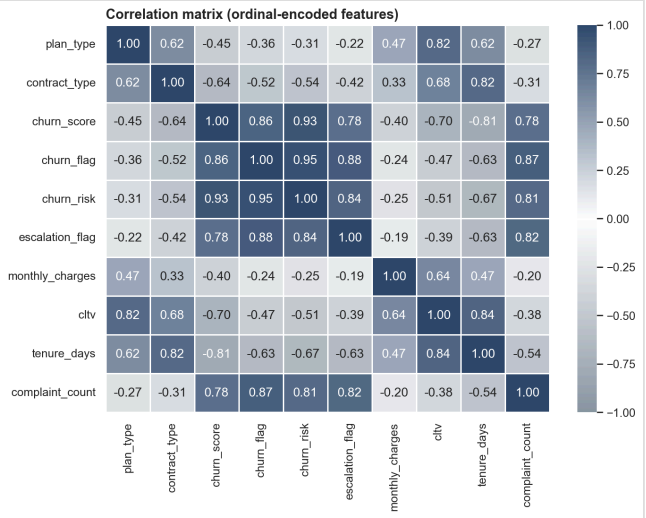


Figure 25. Correlation matrix of ordinal-encoded features

The heatmap uses ordinal encoding (Basic < Standard < Premium, Monthly < Annual, Low < Medium < High). Alphabetical label codes would scramble that order and give misleading correlations. Churn correlates most strongly with complaint count (0.87), escalations (0.88) and churn score (0.86), and negatively with contract length (−0.52) and tenure (−0.63).

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
35-44	7	3	42.9%	Rs 98.93	Rs 2,734
25-34	8	2	25.0%	Rs 136.92	Rs 8,440
45+	4	1	25.0%	Rs 122.96	Rs 3,415
<25	2	0	0.0%	Rs 36.98	Rs 2,705

Segment	Customers	Churned	Churn rate	Monthly revenue	CLTV
Female	12	4	33.3%	Rs 159.88	Rs 5,825
Male	9	2	22.2%	Rs 235.91	Rs 11,469

10. SQL validation

To make sure the pandas logic is right, I recomputed the headline KPIs directly in SQLite. Customers, churned count, churn rate (28.57%), ARPU (Rs 18.85), MRR lost (Rs 73.94) and CLTV lost (Rs 2,047) match the pandas results exactly. The SQL screenshots are in Section 8.

```
SELECT COUNT(*)                                                    AS customers,
       SUM(CASE WHEN cancellation_date IS NOT NULL THEN 1 ELSE 0 END)  AS churned,
       ROUND(100.0 * AVG(CASE WHEN cancellation_date IS NOT NULL
                              THEN 1 ELSE 0 END), 2)                AS churn_rate_pct,
       ROUND(AVG(monthly_charges), 2)                              AS arpu,
       ROUND(SUM(CASE WHEN cancellation_date IS NOT NULL
                      THEN monthly_charges END), 2)                AS mrr_lost,
       SUM(CASE WHEN cancellation_date IS NOT NULL THEN cltv END)  AS cltv_lost
FROM db_subscription;
```

11. Key insights

1. **Contract type is the strongest churn driver.** Monthly subscribers churn at 55.56% against 8.33% for annual subscribers, 6.7 times higher. All six churned customers except one were on monthly contracts.
2. **Support escalations are an early warning signal.** All 5 customers with an escalated complaint churned (100%), compared with 6.25% of everyone else. The correlation between escalation and churn is 0.88.
3. **Basic plan churns most, Premium carries the revenue.** Basic churns at 60%, Standard at 22% and Premium at 14%. Premium contributes 55% of monthly revenue and 63% of total CLTV, so revenue exposure is lower than the headcount churn suggests.
4. **Referral customers are the least loyal.** Customers acquired by referral churn at 83% (5 of 6); paid acquisition churns at 17% and organic at 0%. The referral funnel brings in low-intent users.
5. **We are losing low-value, short-tenure customers.** Average CLTV of churned customers is Rs 341, against Rs 1016 for retained customers (3× gap). The churned group lost Rs 73.94 of MRR (18.68%) and Rs 2,047 of CLTV.
6. **Churn peaked in September 2024 and is concentrated geographically.** Two of six cancellations happened in Sep-2024. Karnataka lost 2 of 2 customers and Meghalaya 2 of 3; five states had zero churn.
7. **Revenue at risk today.** 2 active customers already sit in the Medium-risk tier, with Rs 34.98 MRR and Rs 1,190 CLTV at risk.

12. Recommendations

Action	What to do	Why / expected impact
Monthly → annual migration	Offer monthly subscribers an annual plan with 2 months free at renewal.	Annual churn is 6.7× lower; each converted user removes most of their churn risk.
Escalation SLA	Treat every escalated complaint as a churn alert: senior agent, 48-hour resolution, goodwill credit, follow-up CSAT.	Targets the segment with 100% churn.
Fix the referral funnel	Tighten referral eligibility, extend onboarding and nudge referral users to annual plans before the trial ends.	Referral churn is 83%.
Basic-plan value	A/B test a content bundle or a low-cost upgrade path for Basic users.	Addresses 'Not enough content' and 'Too expensive' cancellations.
Proactive outreach	Contact Medium/High-risk active customers this month (email, SMS, call), ordered by CLTV.	Protects Rs 34.98 MRR / Rs 1,190 CLTV.
Root-cause review	Investigate Karnataka and September 2024 for price changes, outages or competitor launches.	Two cancellations cite a competitor.

How to measure success

- Monthly churn rate and the monthly → annual conversion rate, tracked monthly
- Churn rate among escalated customers (target well below today's 100%) and escalation resolution time
- 90-day retention of referral sign-ups
- MRR and CLTV saved from the at-risk outreach list

13. Limitations and next steps

- **Small sample:** 21 customers is enough to show the method, not to prove significance. Treat the rates as directional and confirm them on the full base.
- **Given churn score:** `churn_score` comes with the data. Next step: train my own churn model (logistic regression or gradient boosting) and compare.
- **Dashboarding:** publish the KPIs in Power BI or Tableau, refreshed from the SQL database.
- **Cohort analysis:** track retention curves by sign-up month once more history is available.

14. Project deliverables

Deliverable	Location
Analysis notebook (step-by-step, with outputs)	notebooks/churn_analysis.ipynb
Reusable pipeline script	src/churn_pipeline.py
Raw data (SQLite database is built on run)	data/raw/customer_churn_data_raw.xlsx
Clean master table and KPI summary	data/processed/
Charts and screenshots	images/charts/, images/screenshots/
This report	reports/Customer_Churn_Analysis_Report.pdf
Stakeholder presentation	presentation/Customer_Churn_Analysis.pptx

Appendix: resume highlights

- Built an end-to-end churn analytics pipeline (Excel → SQLite → Python) for an OTT subscription dataset, cleaning and joining 3 relational tables and computing 20+ KPIs; found a 28.6% churn rate and Rs 73.94 (18.7%) monthly revenue leakage.
- Showed that monthly-contract users churn 6.7× more than annual users (55.6% vs 8.3%) and that every escalated support case ended in churn (corr 0.88), leading to a contract-migration and escalation-SLA retention plan.
- Validated pandas KPIs with SQL (CASE WHEN, JOIN, GROUP BY, HAVING) and presented 10 Matplotlib/Seaborn visualisations and a stakeholder deck with prioritised recommendations.